AquaLLM - Planning

1. Project Context & Rules of Engagement
Context: We have a working 3D aquaculture simulation (AquaOptima) written in Python (backend) and Three.js (frontend). The project uses Particle Swarm Optimization (PSO) to control the fish swarm's behaviors, and Evolutionary Algorithm (EA) to optimize pond management policies (food, probiotic, and oxygen pumping).

Goal: Replace the EA with an LLM-based "Farm Manager" Agent using a Plan-and-Execute architecture. We will run Ablation Studies on this architecture. Rules for Claude:

1. DO NOT modify the core Particle Swarm Optimization (PSO) physics or fish stat decay logic.
2. DO create modular, well-typed Python files for the LLM agent.

Challenge: Find a model that is free or cheap. I need to run it many times to test and develop for this project AquaLLM.

Scope: Need to complete in 1 day. I have a presentation tomorrow. Feel free to remove any part that you think it is too much to do in 1 day.

Similar Work: Generative Agents: Interactive Simulacra of Human Behavior (https://arxiv.org/abs/2304.03442).

2. The Configuration & Ablation Schema
Create a configuration class that will drive our Ablation Studies. This dictates which parts of the "brain" are active.

For example:
python
```
# agent_config.py
from dataclasses import dataclass

@dataclass
class AgentConfig:
    use_observer: bool = True       # True = Text summaries. False = Raw JSON state dump.
    use_memory: bool = True         # True = Keep history. False = Amnesiac (current day only).
    use_reflection: bool = True     # True = Generate periodic insights. False = Raw memory only.
    use_calculator: bool = True     # True = LLM can call budget tool. False = LLM must guess math.
    use_planning: bool = True       # True = Plan-and-Execute (CoT). False = Zero-shot JSON output.
```

3. The JSON Output Schema (The Policy)

The LLM must output its final decision strictly matching this schema. Below is an example, you do NOT have to use it exactly.

```
class PondPolicy(BaseModel):
    food_interval: int = Field(ge=2, le=24, description="Hours between food drops")
    food_quantity: int = Field(ge=5, le=20, description="Pellets per drop")
    food_location: int = Field(ge=0, le=9, description="Drop zone 0-9")
    probiotic_interval: int = Field(ge=24, le=168, description="Hours between probiotic drops")
    probiotic_quantity: int = Field(ge=1, le=5, description="Pellets per drop")
    probiotic_location: int = Field(ge=0, le=9, description="Drop zone 0-9")
    oxygen_interval: int = Field(ge=1, le=24, description="Hours between pump activations")
    oxygen_duration: int = Field(ge=1, le=4, description="Hours pump remains active")
    oxygen_location: int = Field(ge=0, le=9, description="Pump zone 0-9")
```

4. Implementation Phases for Claude

Phase 1: The Perception Module (observer.py)

Write a class PondObserver. It takes the raw simulation state (fish stats, hazard zones, budget) and translates it to a short summary of the pond (only summarize what a “farmer” can observe).

Example:

- If config.use_observer == True: Return a natural language string. Use thresholds (e.g., if avg_fullness < 60, say "The swarm is hungry"). Mention hazard zones (e.g., "NH3 detected in zone 4").
- If config.use_observer == False: Return a truncated, raw JSON string of the numerical arrays.

Phase 2: The Tool Module (tools.py)

Implement the Calculator tool for budget management. Example:

Create a function calculate_daily_cost(policy: PondPolicy) -> float.
Math: (24/food_interval * food_quantity * 0.05) + (24/probiotic_interval * probiotic_quantity * 0.75) + (24/oxygen_interval * oxygen_duration * 2.50).
Format this as a callable tool schema compatible with the LLM API.

Phase 3: The Memory & Reflection Module (memory.py)

Create a MemoryStream class.

Needs a list to store daily observations and the LLM's past actions.
Reflection Logic: If config.use_reflection == True, write a method generate_reflection(llm_client). Every 5 simulated days, pass the last 5 days of memory to the LLM and prompt: "Synthesize this log into a 2-sentence high-level insight about the pond's dynamics." Store this insight.

Phase 4: The Core Agent (agent.py)

Create the FarmManagerAgent class. This is the main Plan-and-Execute loop.

Inputs: AgentConfig, current_state.
Step 1 (Context Assembly): Gather the Observation (from Phase 1). If config.use_memory, append past logs. If config.use_reflection, append the latest insight.
Step 2 (Tool Loop): If config.use_calculator, allow the LLM to output a tool call to check the budget before finalizing.
Step 3 (Prompting):
If config.use_planning == True: Prompt the LLM to write a "Plan" (text reasoning) first, followed by the JSON policy.
If config.use_planning == False: Append "OUTPUT ONLY JSON. DO NOT THINK OR EXPLAIN." to the prompt.
Step 4 (Parsing): Extract the JSON, validate it against the PondPolicy Pydantic model, and return it to the simulation.

Phase 5: The Simulation Hook (sim_hook.py)
Locate the main simulation loop in the existing codebase.

Remove/Bypass the EA generation logic.
Implement a pause every 24 simulated hours (timestep % 24 == 0).
At the pause, call FarmManagerAgent.step(current_state).
Apply the returned PondPolicy to the pond's dynamic variables and resume the simulation.

5. Instructions for Claude
Claude, please begin by analyzing the existing codebase to locate the main simulation loop and the data structures holding the pond state. Remember that everything is just a prototype so far, so feel free to adjust them as you think it would be better (more optimized, more precise…)

First, help me find a free (or cheap) Cloud LLM model that I can use for this project. If it is a Local model, make sure it can run in reasonable time on my MacBook Air M2 or Google Collab Pro. Should I use MacBook Air M2 or Google Collab Pro for this project?

Once you have mapped the state variables, discuss with me about your plan. Stop at each Phase until I have reviewed and approved the plan before moving to the next state.